Title: Threaded BST Predecessor and Successor

1. Problem Statement

To write a C++ program to find and display the inorder predecessor and successor of a specified node in a threaded Binary Search Tree (BST), wherever they exist. The program creates a threaded BST, searches for the given node, and displays its predecessor and successor when available.

2. Objectives

• To understand the concept of a Threaded Binary Search Tree.

• To create a threaded BST by inserting node values.

• To perform inorder traversal using threads.

• To search for a specified node in the tree.

• To find and display the inorder predecessor of a node.

• To find and display the inorder successor of a node.

• To handle cases where the node, predecessor, or successor does not exist.

• To improve programming and problem-solving skills.

3. Algorithm

Step 1: Start the program.

Step 2: Define a Node structure containing data, left and right pointers, and lthread and rthread flags.

Step 3: Initialize the root pointer to NULL.

Step 4: Accept the number of nodes and insert the given values into the threaded BST while maintaining the threads.

Step 5: Display the inorder traversal by starting at the leftmost node and repeatedly finding its inorder successor.

Step 6: Accept the value of the node whose predecessor and successor are required.

Step 7: Search for the specified node. If it is not found, display a message.

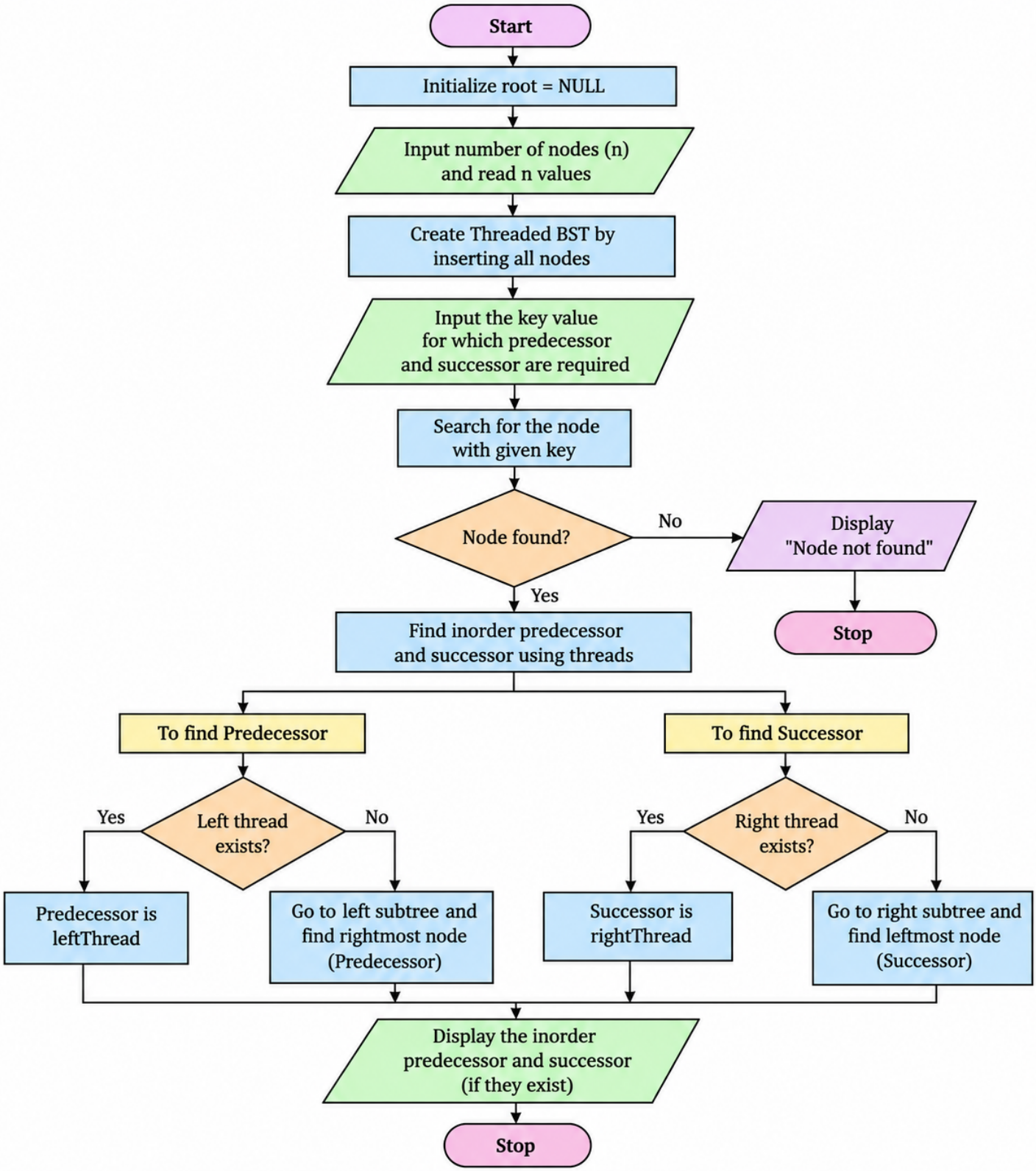
Step 8: If the left pointer is a thread, use it as the inorder predecessor; otherwise, find the rightmost node in the left subtree.

Step 9: If the right pointer is a thread, use it as the inorder successor; otherwise, find the leftmost node in the right subtree.

Step 10: Display the predecessor and successor wherever they exist.

Step 11: Stop the program.

Flowchart – Threaded BST Predecessor and Successor



5. Conclusion

The Threaded BST Predecessor and Successor program demonstrates the practical use of a threaded Binary Search Tree in C++. It uses threads to find the inorder predecessor and successor of a specified node. The program also displays the inorder traversal and handles cases where a predecessor or successor is not available, such as for the smallest or largest node. Overall, it helps in understanding binary search trees, pointers, threads, and traversal.